

Unified Student Ecosystem

Project Specification Document

The Unified Student Ecosystem is a scalable and feature-rich platform designed to facilitate online learning for students, corporate professionals, and educators. It provides a seamless experience for course creation, enrolment, progress tracking, and assessments while ensuring compliance with security and accessibility standards for trainers, students and administrators.

Phase 1: Planning & Architecture (1-2 Weeks)

Objective:

Define project scope, choose the right technologies, and plan the system architecture.

Tasks:

1. Requirement Gathering

- Define all features: user roles, course structure, progress tracking.
- Research competitors (Udemy, Coursera) for inspiration.
- Create a feature list with MVP (Minimum Viable Product).
- **Business & User Needs Analysis:**
 - Identify target users (corporate learners, students, educators, etc.).
 - Understand key pain points this system aims to solve.
 - Ensure compliance with GDPR, ADA for accessibility.

2. System Architecture Design

- Choose Monolithic vs. Microservices (Microservices if scaling is key).
- Define MongoDB schema (Users, Courses, Enrollments, Progress).
- Plan API endpoints (REST/GraphQL).

3. Tech Stack Finalization

- **Frontend:** React.js / Next.js (SSR for SEO).
- **Backend:** Express.js / NestJS.
- **Database:** MongoDB (Mongoose ORM).
- **Storage:** AWS S3 / Firebase (for videos, PDFs).
- **Authentication:** JWT + OAuth (Google, Facebook, GitHub).

4. Wireframing & UI/UX Planning

- Use Figma/Lucid Chart or Adobe XD for wireframes.
- Plan user journeys: Login → Browse Courses → Enroll → Track Progress.
- Accessibility Considerations: Ensure WCAG 2.1 compliance for UI/UX design.

Phase 2: Backend Development (2-4 Weeks)

Objective:

Build the backend with authentication, user roles, and APIs for course management.

Tasks:

1. User Authentication & Authorization

- Implement JWT-based authentication (Register/Login).
- Role-based access control: Admin, Trainer, Learner.
- OAuth for social logins (Google, Facebook).

2. Admin Module

- Admin dashboard to approve/reject courses.
- Manage trainers & learners (CRUD).
- Generate analytics reports (Active users, Course enrollments).

3. Trainer Module

- Upload video courses (AWS S3, Cloudinary).
- Upload PDF/PPT tutorials.
- Edit & delete courses.
- Track student progress.
- Daily status of course
- Quizzes and Assignments

4. Learner Module

- Browse available courses.
- Enroll & start learning.
- Track progress (completed lessons, quizzes).
- Leave ratings & feedback.

Phase 3: Frontend Development (4-6 Weeks)

Objective:

Develop a clean UI/UX for Admin, Trainer, and Learner.

Tasks:

1. UI/UX Design

- Choose Tailwind CSS / Material UI for styling.
- Implement responsive design (Mobile & Web).

2. Admin Dashboard

- View & manage trainers, learners, and reports.
- Approve/reject course uploads.

3. Trainer Dashboard

- Upload & manage courses.
- View enrolled learners & their progress.

4. Learner Dashboard

- Watch video lessons.
- Download PDF/PPT materials.
- Track course progress (lesson completion, quiz scores).

Phase 4: Additional Features & Testing (6-7 Weeks)

Objective:

Add advanced features like payments, analytics, and testing.

Tasks:

1. Progress Tracking & Analytics

- Track user lesson completion.
- Generate reports for admin & trainers.
- **Performance & Scalability Planning:** Implement load testing to handle high traffic.

2. Testing & Debugging

- Unit testing (Jest for backend).
- Integration testing (Cypress for frontend).
- Security testing against XSS, SQL Injection, API abuse.

3. Deployment & Scaling

- Deploy frontend on Vercel / Netlify.
- Deploy backend on AWS / DigitalOcean.
- Set up CI/CD pipelines (GitHub Actions).
- **Disaster Recovery & Backup Plan:**
 - Define data backup strategies (daily, weekly).
 - Implement a recovery plan for server failures.

Phase 5: Post-Launch & Enhancements (Week 8 & Beyond)

Objective:

Improve user experience & add AI-based recommendations.

Future Features:

- AI-based course recommendations.
- Mobile app version (React Native).
- Chat feature between learners & trainers.
- Community forum for discussions.
- Monetization Model: Implement subscriptions, one-time payments, and referral programs.

Microservices Breakdown

1. Authentication Service

- Handles user registration, login, and role-based access.
- Uses JWT + OAuth for authentication.
- Stores user details in MongoDB.

2. Course Service

- Trainers can upload, edit, and manage courses.
- Supports video uploads (AWS S3 / Cloudinary).
- Handles course categorization & metadata.
- Content Approval Workflow: Define an approval system before publishing courses.

3. Content Service

- Manages PDF/PPT tutorials.
- Handles file storage & retrieval.

4. Enrolment Service

- Learners can enrol and unenroll from courses.

- Tracks progress & course completion.

5. Payment Service (If required)

- Manages payments via Stripe / Razorpay.
- Supports one-time payments & subscriptions.

6. Review & Feedback Service

- Learners can rate & review courses.
- AI-based feedback analysis for trainers.

7. Notification Service

- Sends emails & push notifications.
- Alerts users about course updates, enrolments.

8. Analytics Service

- Tracks user progress, engagement, revenue.

Tech Stack & Architecture

Frontend (User Interface)

- React.js / Next.js (for SSR & SEO).
- Tailwind CSS / Material UI (for styling).
- Redux Toolkit / Zustand (for state management).

Backend (Microservices Architecture)

- NestJS / Express.js (for APIs & microservices).
- MongoDB (MongoDB Atlas) (for database storage).
- RabbitMQ / Kafka (for event-driven communication).
- Redis (for caching frequently accessed data).
- Elasticsearch (for search & recommendations).
- API Gateway (Express / GraphQL) (for routing & load balancing).

Storage & Hosting

- AWS S3 / Firebase (for videos, PDFs).
- Vercel / Netlify (for frontend deployment).
- AWS EC2 / Kubernetes (K8s) (for backend microservices).
- Docker & CI/CD (GitHub Actions, Jenkins) (for automated deployments).

Security

- JWT & OAuth (Google, Facebook) for authentication.
- Role-Based Access Control (RBAC) for access management.
- AWS WAF / Cloudflare for DDoS protection.